Simulación de administración de memoria: Second Chance y Aging

Tarea 5 · Sistemas Operativos · Equipo E04

David Rivera Morales Yessica Vianney Montes de Oca Aguilar

octubre de 2026

Agenda

1. El problema: fallos de página y reemplazo
2. Second Chance y Aging
3. Metodología y validación
4. Demo en vivo
5. Resultados y análisis
6. Conclusiones

El problema

- La memoria virtual divide el proceso en **páginas**; la memoria física en **marcos**.
- Referencia a una página no residente $\Rightarrow$ **fallo de página**: hay que traerla y quizás expulsar otra.
- El algoritmo de reemplazo decide la víctima; su calidad se mide con la **tasa de fallos** sobre la cadena de referencias.
- Las cargas reales tienen **localidad**: conviene recordar qué se usó recientemente.

¿Por qué aproximar LRU?

- LRU es bueno, pero caro: cada referencia actualiza el orden de uso.
- FIFO es barato, pero malo: expulsa la más antigua sin mirar el uso.
- Second Chance y Aging son dos puntos del compromiso metadata/costo.

Algoritmo	Metadata por marco	Aproxima a
Second Chance	1 bit de referencia	LRU / FIFO híbrido
Aging	N bits (8 por defecto)	LRU con historia finita
FIFO	0 bits	–
LRU	orden de uso	exacto
OPT	el futuro	óptimo teórico

Second Chance: idea y pseudocódigo

- Lista circular de marcos + puntero `hand` + bit R por marco.
- Acierto: $R = 1$. Fallo: el reloj limpia bits hasta hallar un 0.

```
si p está en memoria:
    R[p] = 1
si no:
    mientras R[hand] == 1:
        R[hand] = 0
        hand = (hand + 1) mod F
    víctima = memoria[hand]
    memoria[hand] = p
    R[hand] = 1
    hand = (hand + 1) mod F
```

Second Chance: ejemplo paso a paso

Traza clásica, 4 marcos, estado `página:R`:

Paso	Ref	Resultado	Estado	Víctima
5	0	acierto	`7:1 0:1 1:1 2:1`	–
6	3	fallo	`3:1 0:0 1:0 2:0`	7
7	0	acierto	`3:1 0:1 1:0 2:0`	–
8	4	fallo	`3:1 0:0 4:1 2:0`	1

- Paso 6: todos los bits en 1 $\Rightarrow$ vuelta completa, 4 bits limpiados, 5 marcos visitados, expulsa al 7.
- Paso 8: solo limpia el bit de la página 0 y expulsa al 1.

Second Chance: costo y Belady

- Costo: 1 bit por marco; 2.32 visitas por fallo en `localidad` (2271 marcos examinados, 1293 bits limpiados).
- Si todos los bits están en 1, el reloj limpia y expulsa donde empezó; si eso pasa en cada reemplazo, avanza como FIFO (no siempre).
- No es algoritmo de pila: exhibe la anomalía de Belady ($9 \rightarrow 10$ fallos de 3 a 4 marcos).

Aging: idea y pseudocódigo

- Contador de N bits por marco; el MSB pesa más que los bits viejos.
- Cada referencia envejece todo; la víctima es el contador mínimo.

```
para cada página residente q:
    age[q] = age[q] >> 1
si p está en memoria:
    age[p] = age[p] | MSB
si no:
    víctima = argmin(age[q])
    memoria[víctima] = p
    age[p] = MSB
```

Aging: convención de esta simulación

- Un paso de envejecimiento **por referencia**, no por intervalo.
- Desplazamiento **antes** de elegir víctima; uso actual al final.
- No hay reloj de muestreo de R como en el Aging periódico de Tanenbaum (que muestrea R cada k ticks y lo limpia).
- Los resultados corresponden a esta convención; no comparables directamente con otra frecuencia de actualización.

Aging: ejemplo paso a paso

Traza clásica, 4 marcos, estado `página:edad` en hex:

Paso	Ref	Resultado	Estado	Víctima
5	0	acierto	`7:08 0:90 1:20 2:40`	–
6	3	fallo	`3:80 0:48 1:10 2:20`	7 (04)
7	0	acierto	`3:40 0:A4 1:08 2:10`	–
8	4	fallo	`3:20 0:52 4:80 2:08`	1 (04)

- Paso 6: 08→04, 90→48, 20→10, 40→20; víctima 7 con 04; la página 3 entra con 80.
- La página 0 se usó en el paso 5: su contador 90 la protege.

Aging: por qué importan los bits

- N bits = ventana de historia de N referencias.
- Con 8 bits y memorias grandes (9–12 marcos) las páginas viejas empatan en cero y se pierde el orden de recencia $\Rightarrow$ Second Chance gana.
- Con 14–16 bits Aging recupera terreno frente a LRU.
- Costo: N bits por marco, $O(F)$ por referencia (desplazar + buscar mínimo).

Metodología: trazas

Traza	Refs.	Páginas	Propósito
`clasica`	20	6	caso de libro (páginas 0,1,2,3,4,7)
`belady`	12	5	anomalía de Belady
`ciclica`	100	10	peor caso: ciclo de 10 páginas
`localidad`	2000	24	carga principal: 8 fases, 85 % caliente
`mixta`	1200	20	segunda carga: 70 % caliente

- Simulación discreta dirigida por trazas, determinista (semillas fijas).
- Métricas: fallos, aciertos, expulsiones, residencia media, costo del reloj y de los contadores.

Validación

- **24 self-tests** con valores de libro (Silberschatz, Belady, reloj, decaimiento, estrés): 0 fallos.
- **1440 comparaciones** contra una implementación independiente en Python: 0 diferencias.
- **13 regresiones** del lector de trazas.
- Invariantes por corrida: fallos + aciertos = referencias, sin páginas duplicadas, marcos examinados = bits limpiados + fallos con memoria llena.

Demo en vivo

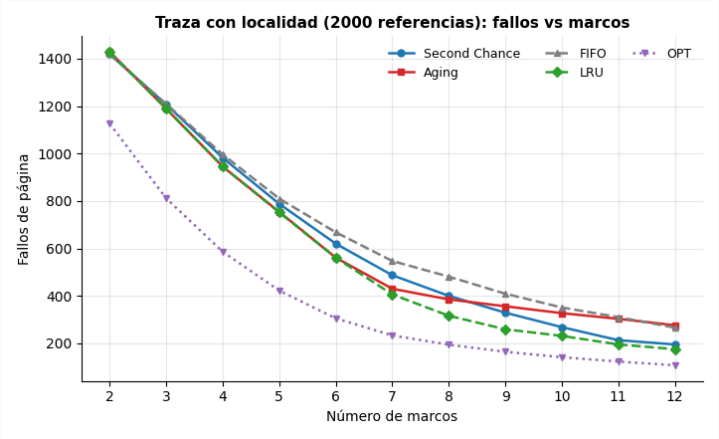
```
make test                                          # 24 comprobaciones
./simulador -m 4 traces/localidad.txt              # los 5 algoritmos
./simulador -a second -m 4 -t traces/clasica.txt # reloj paso a paso
./simulador -a aging  -m 4 -t traces/clasica.txt # contadores en hex
./simulador -m 3 traces/belady.txt                 # 9 fallos (FIFO/SC)
./simulador -m 4 traces/belady.txt                 # 10 fallos: anomalia
./simulador -m 9 traces/ciclica.txt                # Aging rompe el ciclo
```

Resultados: 4 marcos

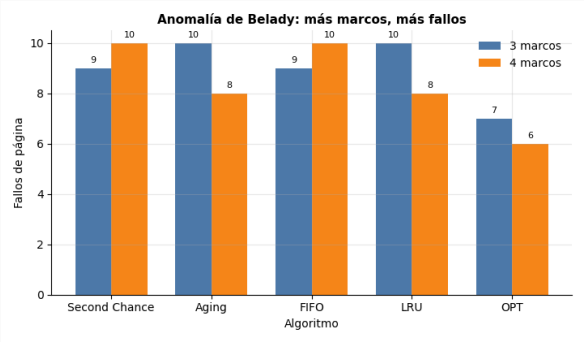
Traza	SC	Aging	FIFO	LRU	OPT
`clasica`	9	**8**	10	**8**	**8**
`belady`	10	**8**	10	**8**	**6**
`ciclica`	100	100	100	100	**70**
`localidad`	982	**945**	996	**945**	**586**
`mixta`	660	**638**	673	**638**	**406**

- Con 4 marcos, Aging empata con LRU y le saca 37 fallos a Second Chance en `localidad`; OPT marca la cota teórica.

Resultados: cruce alrededor de 9 marcos

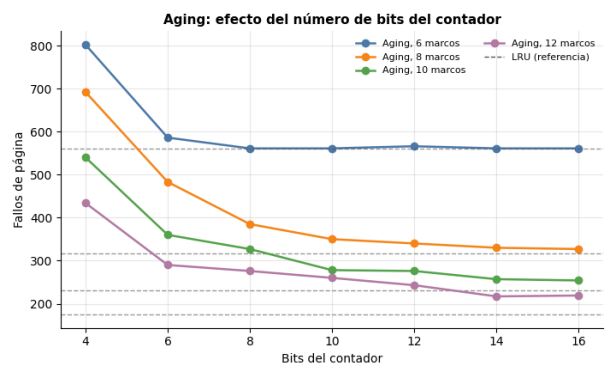


Resultados: anomalía de Belady



- SC y FIFO: 9 $\rightarrow$ 10 fallos al pasar de 3 a 4 marcos.
- Aging, LRU y OPT bajan, como se espera.

Resultados: sensibilidad de los bits



- 4 bits: lejos de LRU.
- 8 bits y 6 marcos: empata con LRU (561).
- 14–16 bits: se acerca a LRU con 8–12 marcos.

Resultados: la traza cíclica

Marcos	SC	Aging	FIFO	LRU	OPT
2–8	100	100	100	100	90→30
9	100	**65**	100	100	20
10	10	10	10	10	10

- Con 9 marcos, FIFO, LRU y SC caen en el peor caso del ciclo $F + 1$.
- Aging elige por contador, no por antigüedad, y rompe el ciclo.
- Caso límite llamativo, pero aislado: no una garantía general.

Análisis (1)

- Aging es el mejor de los dos con 4 marcos: empata con LRU y reduce los fallos de Second Chance en 3–4 %.
- Second Chance es más barato: un bit por marco y 2.32 visitas por fallo; Aging desplaza F contadores por referencia.
- El tamaño de la memoria cambia el ganador: a partir de 9 marcos gana Second Chance; el contador de 8 bits se queda corto de historia.

Análisis (2)

- Second Chance nunca es peor que FIFO en estas trazas, pero sí sufre la anomalía de Belady.
- Aging no muestra la anomalía aquí, pero tampoco es un algoritmo de pila: no se puede prometer que siempre baje al agregar marcos.
- OPT marca el margen: 586 contra 945 fallos de Aging deja un 38 % de mejora teórica no alcanzable en línea.

Conclusiones

- Second Chance compra mucho con un solo bit; Aging compra más precisión con ocho bits y más trabajo por referencia.
- El ganador depende del tamaño de memoria y de la localidad de la carga, no del nombre del algoritmo.
- OPT es una cota, no una opción; FIFO es el piso de calidad.
- La validación (24 self-tests + 1440 comparaciones) asegura que las diferencias son del algoritmo, no de la implementación.

¿Preguntas?

Equipo E04. Simulador en C11, trazas versionadas y resultados reproducibles con `make test`